

Another, Doubtless Very Different, Bill James: Baseball Numbers and Software Numbers

Alex Groce

Northern Arizona University

Flagstaff, USA

agroce@gmail.com

Abstract

Software engineering metrics and baseball metrics have a lot in common; this essay argues that we can make use of the common features, with due attention to the critical differences, to guide future work on software engineering metrics.

CCS Concepts: • Software and its engineering → Dynamic analysis; Software testing and debugging.

Keywords: software metrics

ACM Reference Format:

Alex Groce. 2026. Another, Doubtless Very Different, Bill James: Baseball Numbers and Software Numbers. In *Proceedings of the 2026 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward! '26)*, October 3–9, 2026, Oakland, CA, USA. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3689492.3689810>

1 Introduction

Harlan D. Mills (1919-1996) is the namesake of the Harlan D. Mills Award given by the IEEE Computer Society to recognize “researchers and practitioners who have demonstrated long-standing, sustained, and meaningful contributions to the theory and practice of the information sciences, focusing on contributions to the practice of software engineering through the application of sound theory.” First given in 1999, recipients include such luminaries as David Parnas (the first winner), Elaine Weyuker, Bertrand Meyer, the Cousots, Gerard Holzmann, Pamela Zave, Gail Murphy, Mark Harman, Matt Dwyer, David Harel, and, most recently, Andreas Zeller. Mills’ idea of the Chief Programmer Team influenced Brooks’ proposals for how to avoid software disaster in *The Mythical Man-Month*; his Cleanroom conception informs every approach that is focused on defect-prevention (from static

analysis to aggressive unit testing to mutation testing) rather than removing defects found in the field; Mills was also a major force in introducing both statistics (viewing tests as statistical experiments, which arguably also is a foundational insight for all of random testing and fuzzing) and formal specification (thus a contributor to the lines of thought that have given the field Alloy, TLA+, Dafny, and other treasures). In short, Mills was a genuinely major figure in our field, though (other than having one of the top awards in the field named after him) I think he’s one of the least talked-about major visionaries we have. Indeed, one name for the Mills Award, confusingly implied as being started in 1996, and perhaps by ICSE/the ACM, in Wikipedia, is the “Harlan Mills Practical Visionary Prize”.

I know the rough outlines of the above details because I am a software engineering history buff. But I also know that Mills was a command pilot for the US Army Air Corps in WWII; that he rewrote the B-24 pilot training program because he thought it was garbage; and that he worked at the Institute for Advanced Study alongside Einstein, Gödel, and von Neumann. I know these things because Harlan Mills and his brother Eldon are the most prominent characters in Chapter 4 of Alan Schwarz’s book, *The Numbers Game: Baseball’s Lifelong Fascination with Statistics* [8]. They appear there because, besides having a passion for applied mathematics, operational research, and software engineering, Harlan Mills had a passion for baseball. In 1968, Harlan and Eldon Mills founded a company called Computer Research in Sports. Their goal was to go beyond the statistics baseball fans already knew to statistics that truly measured how a player contributed to winning a game. For example, they wanted to look beyond the much-beloved batting average, a statistic we will talk about below that, one could argue, drove the country crazy a few times in the years since baseball became America’s pastime. Harlan and Eldon were among many high-powered intellects (including multiple Nobel Prize winners and the popular Harvard evolutionary-biology maverick, public intellectual, and superb writer Stephen Jay Gould) pioneering an effort to truly understand baseball, an effort that is documented in Schwarz’s engaging and in-depth look at baseball’s love of statistics and how those statistics have developed from the earliest, alien-looking, box-scores.

The connection between baseball statistics and software engineering, however, goes beyond the curiosity that Mills was a pioneer in both software engineering and baseball

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org. *Onward! '26*, October 3–9, 2026, Oakland, CA, USA

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1215-9/24/10

<https://doi.org/10.1145/3689492.3689810>

stats-savvy. The obvious connection is that the stats revolution in baseball was in substantial part tied to the computer revolution; nothing like <https://www.retrosheet.org/> could plausibly exist in a non-computerized world. This site, which offers play-by-play accounts of over 100,000 pro baseball games makes it possible to search for trends or individual facts, a task that would be impossible even if a warehouse of filing cabinets had the relevant data. Schwarz's book shows how computers were essential to the growth of the modern baseball stats world, and takes us back to when baseball work was done by "stealing" off-time on hulking company and government mainframes. The deeper connection, however, is that baseball was, for a very long time, tied to extremely traditional, and beloved, statistics, but even stodgy, change-averse professional baseball front and back offices eventually realized the grave limitations of these statistics. Once they made this realization, they started making use of more sophisticated, nuanced, statistics that truly reflected how to win the game. That story: famous, widely-used numbers, lacking in proven operational relevance or at least clearly sub-optimal in definition, should sound familiar to students of software engineering.

2 Build a Better Mousetrap/Metric

Perhaps we can quickly see what I mean by the move from traditional stats to ones that really do *mean more* by looking at a simple example from software testing, paired with the most famous example from baseball (and the only one I think I can explain tolerably well to software people who don't know baseball).¹

One way to determine if a program is well-tested is to take the test suite for the program and look at the statement coverage: the percent of all statements in the whole program that are executed at least once by the test suite. Obviously if a test suite fails to execute most of the code in the program it is ostensibly testing, it is likely a pretty bad test suite! Similarly, if a test suite runs 90% of the code in the program, it's presumably at least doing something non-trivial right.

In baseball, for a very very long time, the king of statistics was batting average. Even if you know little about baseball, even if you're a European or from Mars², you probably know that in baseball, a batter is thrown balls by the pitcher and tries to hit those balls, and then (when the ball is hit) run to a series of three bases, before returning to the place from which the ball is initially hit (home base). Batting average looks at how often a player hits the ball to advance to first, second, third, or home base. Batting average is not nonsense! The most sophisticated baseball statisticians tend to agree players celebrated for their batting averages, such as Ty Cobb,

Rogers Hornsby, and Ted Williams, were superb, top-of-the-line players; the top players by batting average now, like Aaron Judge, tend to be household names for a good reason. Batting average, like code coverage, is a *meaningful number*.

However, there are some problems with statement coverage, and code coverage in general. For one thing, maybe you only run a really important line of code once in the whole test suite, and that in a rather boring and uninteresting situation where the things the line of code is supposed to do are not seriously stressed (it's a complex multiplication, but one factor is zero, for example). Or, worse yet, you may run lots of code lots of times, but not check anything much about whether the program did what it was supposed to do other than "not crash." There are lots of ways a program can go wrong other than outright crashing. Mutation score [4, 6] asks "what do we really care about in testing? We want to find bugs. Statement coverage is an awful roundabout way to measure finding bugs, since bugs aren't even mentioned here. Let's see how a test suite does at finding bugs." In mutation testing, a large number of essentially random bugs are actually injected into a program, and the score is the percent of the fake bugs the test suite was able to find. It's not perfect, but it completely removes the two obvious problems with statement coverage mentioned above, by focusing on *finding the bug*: if you only run code in an uninteresting context, you won't find a bug where the non-zero operand to multiplication is wrong; and you won't find any bugs whose consequences you don't examine in your tests.

Similarly, the way batting average is defined, it doesn't look at every time a player comes up to home plate to try to hit the ball. The numerator is "times the player hit the ball and got to a base" but the denominator is "times the player came up to bat, minus times the pitcher did such a bad job that the player was allowed to take a base without hitting the ball, and the batter took proper advantage of this by not swinging at bad pitches (or times the pitcher feared the player so much this was intentionally done to keep the bat off the ball), times the pitcher hit the player with the ball, times the catcher did something weird, times the player intentionally hit the ball in a way that tends not to let you reach base but has another game-winning-relevant purpose, etc. etc. etc.). On-base-percentage, a revolution in the 1980s in understanding how to value hitters, counted most everything, and looked (essentially) at "of times this player came up to bat, how often did this player end up at least standing on a base." Like mutation score, it focused on the *more* essential (not getting out before reaching a base, finding bugs) rather than the easily and obviously measured (hitting the ball, running the code). The typical non-baseball-fan probably still knows batting average, not on-base-percentage; the typical software developer (who *should* be more like a baseball fan than someone who isn't sure if Babe Ruth and Willie Mays played at the same time or not), alas, probably is still more familiar with code coverage than mutation score,

¹We could also compare LOC produced to modern software productivity measures that are less simpliminded, like DevOps' Change Failure Rate [5].

²The best kid's book on baseball is Walter R. Brooks' *Freddy and the Baseball Team from Mars*, by the way).

but the game is changing, and highly visible adoption of mutation at, for example, Google [7] and Meta [2] suggest a revolution in this metric is underway.

3 Why It Isn't Easy

The above suggests that improving software engineering metrics is easy: just do what the baseball people did, and look at whether the most widely used numbers are measuring the most important things.

In fact, before we start on this, let's just start off with ways software engineering is like baseball, and ways software engineering is not like baseball. Even if you don't know baseball, I am hoping this list will be informative.

3.1 Ways Software Engineering is Like Baseball

- Baseball and software engineering are both, when played for keeps by pros, cases where 1) it is fundamentally a team sport, and no one player, however good, can compete in the pros without a good team but 2) individual ability is extremely important, and the gap between the average player and the best is large and really matters.
- The “game” is deeply structured and appeals to stats-minded thinkers.
- Some of the best books in both fields are “high end practitioners analyzing how they do their magic”: for baseball, these range from Mathewson's legendary *Pitching in a Pinch* to Hernandez's *Pure Baseball*, with in between like Bob Gibson and Bob Gibson + Reggie Jackson (*Pitch by Pitch* and *Sixty Feet Six Inches*). For software, we have legendary books like Brian Kernighan and Rob Pike's *The Practice of Programming* and McConnell's *Code Complete*, as well as *The Pragmatic Programmer*, all the “Uncle Bob” *Clean X* books, and so forth, including, arguably, much of Knuth's work.
- Crazy things that don't fit any model *will* happen; you can understand and even in theory predict a phenomenon like the adoption of smarter metrics by management in baseball, or software companies adopting very good practices with respect to CI/testing; you can't predict a rare genius tech lead taking over a team and producing outsized results, or a couple of years of Billy Martin at the A's in the early 80s turning stolen bases and hit-and-run into remarkably good-for-the-talent seasons. Some things are not in any model.

None of these things contradict a narrative where “doing what on-base-percentage does to batting average, but in more places” is a fruitful and relatively simple path forwards for software metrics. The problem is in the differences, and in fact in a few core differences between baseball and software, bringing us to:

3.2 Ways Software Engineering is Not Like Baseball

- Baseball has a profoundly binary structure: games are won or lost, and each game consists of a set of events with similarly clean, discrete outcomes: an at-bat is generally a “win” for the pitcher or the batter (perhaps with degrees of win for the batter); an inning has a score that is a set of evaluations of scoring opportunities; even something as intrinsically complex and nuanced and hard-to-evaluate as fielding has obvious binary outcomes: was an error handed out? was a potential double play converted?
- Baseball's outcomes are almost entirely *evaluated immediately*: you do not need to wait six innings, much less a season, to know that a batter striking out is bad for the batter's team and good for the pitcher's team. A design decision or API change often will not make its positive or negative payoffs clear for years or even decades. It's nonsensical to think of a baseball player who looks great on paper, racking up runs or outs for a team, but who in the long term has left such a load of baseball's version of technical debt that the player is a hidden liability, not an asset.

Notice that unlike with the similarities, I haven't bothered to discuss the “software side” of the issue, except implicitly: everyone likely to read this knows that most software actions and decisions don't have a simple, binary outcome, and that whatever outcome they do have is often greatly temporally delayed: even if a line of code is simply and utterly wrong, the detection of that wrongness may wait hours, days, months, years, or even never come, if the bug is obscure and unlikely enough to be triggered. If a guy in the outfield fails to catch a ball that should have easily been in his hands, it's clear; there are no subtle misses or catches.

This presents a fundamental problem: advanced statistics in baseball were, most famously, used to let a team without “Yankees-money” (for software-only people: read “Google-money”) snap up undervalued players and perform tricks based on understanding of the deeper, better, statistics. The *Moneyball* movie, which stars Brad Pitt³, has a famous bit of dialogue, where Billy Beane (the hero of the canonical “applying advanced stats to actually winning ball games” story), played by Pitt, talks to other people in the management structure of his team, who are trying to replace a certain critical individual player. Beane says this is a mistake. He adds up the on-base-percentages of three players, and averages them, then says the goal needs to be not to replace any individual player, but to “re-create him... in the aggregate” and find three (ideally, under-valued) players whose on-base-percentage matches the three guys now missing from their team; not reproduce the expensive, hard-to-get, best of them, but match the average of the three, a much easier task. In baseball, this

³That you will not see an equivalent software movie starring Brad Pitt is part of the point of this essay.

pretty much works, if you know the values to use: using the highly-relevant on-base-percentage vs. much more famous, much more lossy, batting average, matters here. It's okay to take a guy who gets a lot of walks, for example: he's good at knowing when to hold off swinging at the ball, which is extremely useful for winning games; that this excites fans less than hitting the ball is ok, because winning games *really* excites fans.

In software, if you lose three developers, you cannot think about a formula for the “good-code-percentage” of the three, and try to match the mean value, with lesser-paid developers, none of whom match the star developer, but who, together, have the same rationally-measured contribution to software success. Your departing developer has too few “at-bats” with too few binary “good/bad” outcomes to turn into a real number; your possible replacements, not being highly (over-?) paid superstars, likely have even less reliable data; furthermore, knowing if any decisions any of these people have made were really good may take years; if you are a startup, you likely will be bought or have gone out of business before it's known if these players hit or struck out on their last at-bat, if it is ever known, which is unlikely.

The software game is just fundamentally hard to score; is Xerox's Palo Alto team a winning or losing team? They changed the world and gave us modern software; they didn't make much money for Xerox. There is no baseball equivalent, really. If you get a pennant or a World Series title, you got it; nobody else gets to scoop up the rewards of genius for what, on paper, is a losing season for you.

To emphasize this point, it's important to note that the most important decisions in software are seldom individual lines-of-code decisions, which map most naturally onto baseball's at-bats; the most important decisions are design decisions, over-arching choices like “do we use microservices or not?” or “what language should we write this in?” at the highest level, or “should this class handle this specialized scenario, or break it out into a separate class” at a lower level. Baseball has important high-level decisions; a manager or owner may prioritize hitting over pitching, power over speed, contact hitting over power, etc., but these decisions still have quicker, clearer evaluation payoff, and are more clearly constrained by budget or existing team, than the most difficult, unconstrained, new-software-system architectural decisions. When you never make the playoffs, despite having a team of very good players, in five years, you can at least strongly suspect you chose wrong (and if you are a manager, the owner will clarify things by firing you, probably); when you made an architectural choice and you're winning over five years on deployment flexibility, but debugging complexity is a nightmare costing you money, reputation, and burnt-out devs, you often can't even guess if you made the right choice; it might well have gone worse the other way.

4 How to Win the Game: Pick the Right Game to Play

In baseball, if you have very bad pitchers, and no way to fix that, you need to play the Boston Red Sox game of racking up a ton of runs with an incredible offense; if you have great pitchers and no offense, you need to play that game. You may not have the ability to play the ideal game (great pitching, great hitting, utter dominance) unless you are the Yankees and have infinite money.

Similarly, if we software engineering thinkers want to up our metrics game, we are going to have to look at where we can compete with baseball in terms of developing real, advanced, meaningful statistics, and where the structure of the game means we need to give up.

First, we have positive examples. Mutation score is not a perfect way to tell if a test suite is good or bad, but it is a much more powerful approach to making a rational guess at that question than code coverage alone, because it focuses on a genuinely meaningful binary statistic in the same way as on-base-percentage: does this test suite detect this example of a bug? This is repeated over a lot of at-bats, and gives us a real measure of something important about a test suite (the power it has to detect real, if hypothetical, bugs).

The above analysis suggests that trying to develop meaningful statistics to evaluate individual players is, at minimum, going to be vastly more difficult in software than in baseball. Baseball has, in fact, exploited the binary nature of a large number of immediately evaluated events to produce sophisticated player statistics based on concepts such as “the delta between a given player and an aggregate average, ‘replacement’ player at the same position” (WAR, Wins Above Replacement), runs created (a clever statistic invented by Bill James, about whom, more in a little bit), an attempt to actually measure the genuine contribution of an individual player to a team's overall set of winning games, accounting for the player's individual role in play (win shares, again a Bill James creation), and Pythagorean winning percentage (again, James, and you mathy folks who don't know baseball can possibly even guess what this one looks like).

The baseball analogy, though it does not give us a way to evaluate individual developers well, does immediately expose some bad metrics. Evaluating developers by lines of code is like evaluating batters by how often they swing at a pitch. The pitch may have been a ball; the swing, in itself, means nothing. What matters is getting on base; in software, getting on base is writing useful, correct code that does not increase technical debt or cognitive load (or at least does not increase it more than is reasonable for the payoff). And often, one swing is better than three; one line is better than five.

One might imagine that individual player evaluation must be left to word-of-mouth, previous salary, resume scanning, and interviews focusing on arbitrary and rather silly puzzles

and coding questions, but that perhaps software is doing a good job at project-level estimation, where the hard-to-measure differences of individual players are more evened-out. Alas, even very well-intentioned approaches such as CO-COMO II [3] have a much higher error rate (30% or worse) than the current ability of Vegas oddsmakers or amateur statisticians to predict the win-loss record of a team for an upcoming season. Large software projects remain notoriously over-budget and over-schedule in a way dating back to Fred Brooks; baseball teams sometimes surprise, but are fairly predictable in a way software projects simply are not.

The way to play is to give up on the games you can't win right now. In the long run "individual developers" may in fact have reliable statistics, if those developers are not humans with limited, hard-to-evaluate, decisions but are LLMs that can be applied to varieties of problems with known solutions for a huge number of "at-bats" and measured objectively on past problems, without undue cost. But for now, most developers are human, and there is no shortcut to evaluating them; similarly, while some effort at project budget and schedule estimation is necessary, it is unreasonable to expect high precision much beyond what is obtained by simply asking expert developers who are given a reasonable outline of the project scope.

But this does not mean that software has nowhere to go in improving the quality of its statistics; it means that we have to look for places where wins are possible, even if those are not the most desirable statistical tools (developer evaluation tools for hiring and firing, project estimation tools for planning). Testing, and more generally, CI, shows the way: because testing has many binary outcomes immediately evaluated, successful/failed compilations, tests passed/failed, and static analysis reports, it can be, and has been, turned into sophisticated statistical measures that offer real power to smart managers and developers. Analysis in these settings has the ability to do real historical examinations too, because old check-ins can be re-tested, and you can actually ask if a proposed measure would have predicted the eventual presence or absence of bugs in code [1].

There are other places where such efforts seem like they might pay off:

- Build and CI pipeline health in general (lots of discrete events, determination of flaky tests, etc.): real time series data
- Dependency and API stability: when a library API breaks downstream consumers, that is often an observed event; vulnerabilities requiring critical patches are big-time, observed, "strike-outs" or "errors"
- Code review turnaround (are bugs detected in code that passed code review, within a short time frame); this one can potentially point fingers at individual developers who may be good developers but are poor reviewers
- Incident/outage attribution (hard, the causality chains are a lot more opaque than the most complex baseball play, but not obviously hopeless)
- And, as noted, LLM/AI developers, who can be put through evaluation exercises that human developers cannot, without undue cost, undergo

These are simply the first few that come to my mind, and really I don't know anything about anything but testing, myself.

5 A New Bill James

In 2006, Time magazine named Bill James one of the 100 most influential people in the world. He worked for the Boston Red Sox for 17 years, during which they won four World Series championships. He has undergraduate degrees in English and economics. And, most importantly, by heroic effort, while a security guard at a pork-and-beans cannery, he asked a bunch of hard questions and gave data-based answers, focusing on things that could actually be answered given the data, and this work, after he wrote it up and fought his way through skeptical editors, kicked off a real revolution in baseball statistics, finally fulfilling the dream of Harlan Mills and other thinkers, by making baseball front offices turn their attention to numbers that meant something and were built for practical use.

James' virtues, in baseball's "numbers game," were a stubborn attention to the solvable problems at hand, a willingness to toil in obscurity until he had something worth saying, and the capability to write up his good ideas in a way that found a large audience, eventually including an audience consisting of the decision makers behind professional baseball teams.

At the end of his landmark book lamenting the collapse of moral philosophy as a vital program capable of directing human activity, the philosopher Alasdair MacIntyre wrote "We are waiting not for a Godot, but for another—doubtless very different—St. Benedict." Software engineering awaits another—doubtless very different—Bill James.

References

- [1] Iftekhar Ahmed, Rahul Gopinath, Caius Brindescu, Alex Groce, and Carlos Jensen. 2016. Can testedness be effectively measured?. In *Proceedings of the 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering, FSE 2016, Seattle, WA, USA, November 13-18, 2016*, Thomas Zimmermann, Jane Cleland-Huang, and Zhendong Su (Eds.). ACM, 547–558. <https://doi.org/10.1145/2950290.2950324>
- [2] Moritz Beller, Chu-Pan Wong, Johannes Bader, Andrew Scott, Mateusz Machalica, Satish Chandra, and Erik Meijer. 2021. What It Would Take to Use Mutation Testing in Industry - A Study at Facebook. In *International Conference on Software Engineering: Software Engineering in Practice (ICSE '21)*. IEEE, 268–277. <https://doi.org/10.1109/ICSE-SEIP52600.2021.00036>
- [3] Barry Boehm, Chris Abts, A. Winsor Brown, Sunita Chulani, Bradford K. Clark, Ellis Horowitz, Ray Madachy, Donald Reifer, and Bert Steece. 2000. *Software Cost Estimation with COCOMO II*. Prentice Hall, Upper Saddle River, NJ.

- [4] R. A. DeMillo, R. J. Lipton, and F. G. Sayward. 1978. Hints on Test Data Selection: Help for the Practicing Programmer. *IEEE Computer* 11, 4 (Apr 1978), 34–41. <https://doi.org/10.1109/C-M.1978.218136>
- [5] DevOps Research and Assessment (DORA). 2014–2024. State of DevOps Reports (various years). <https://dora.dev/>. Defines “change failure rate” as the percentage of production changes that result in degraded service or require remediation (e.g., rollback, hotfix, patch).
- [6] Alex Groce, Mohammad Amin Alipour, and Rahul Gopinath. 2014. Coverage and Its Discontents. In *Proceedings of the 2014 ACM International Symposium on New Ideas, New Paradigms, and Reflections on Programming & Software* (Portland, Oregon, USA) (*Onward2014*). Association for Computing Machinery, New York, NY, USA, 255–268. <https://doi.org/10.1145/2661136.2661157>
- [7] Goran Petrovic and Marko Ivankovic. 2018. State of Mutation Testing at Google. In *International Conference on Software Engineering: Software Engineering in Practice (ICSE '18)*. 163–171. <https://doi.org/10.1145/3183519.3183521>
- [8] Alan Schwarz. 2004. *The Numbers Game: Baseball's Lifelong Fascination with Statistics*. Thomas Dunne Books, New York.